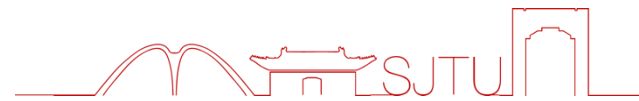




上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

任锐

2026/9/18

饮水思源 · 爱国荣校



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

CI/CD

任锐

—— 饮水思源·爱国荣校 ——





01

Introduction of CI/CD



- **Continuous Integration**
 - The practice of merging all developer working copies to a shared mainline several times a day
 - “The practice where members of team integrate their work frequently; usually each person integrates at least daily – leading to multiple integrations per day.” – Martin Fowler





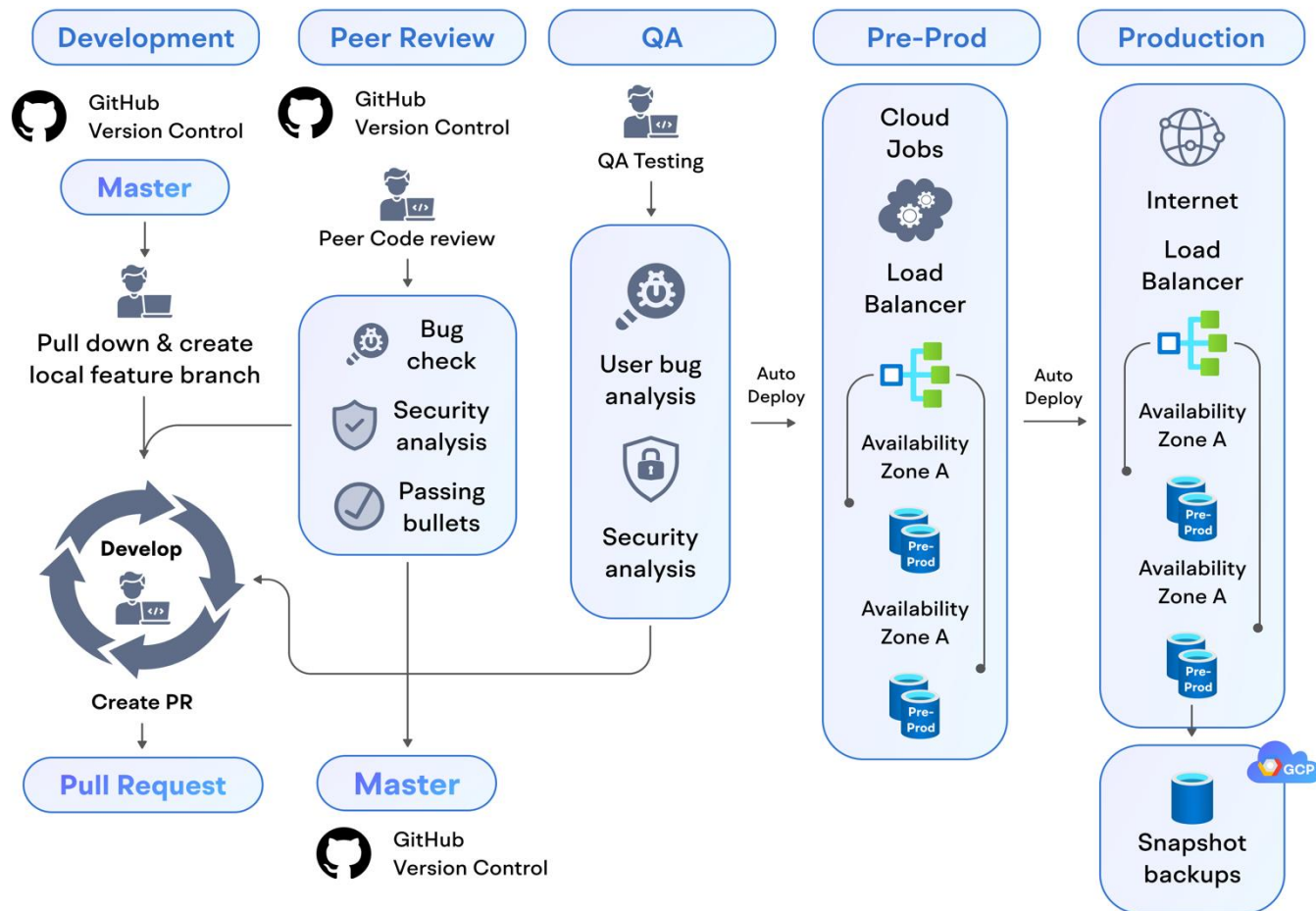
- Continuous Delivery
 - “A software discipline where you build software in such a way that software can be *released to production at any time.*” – Martin Fowler
 - “*Every changes* you make *automatically* get deployed through the *deployment pipeline.*” – Martin Fowler
 - A software engineering approach in which teams produce software in short cycles, ensuring that the software can be reliably released at any time.





Introduction

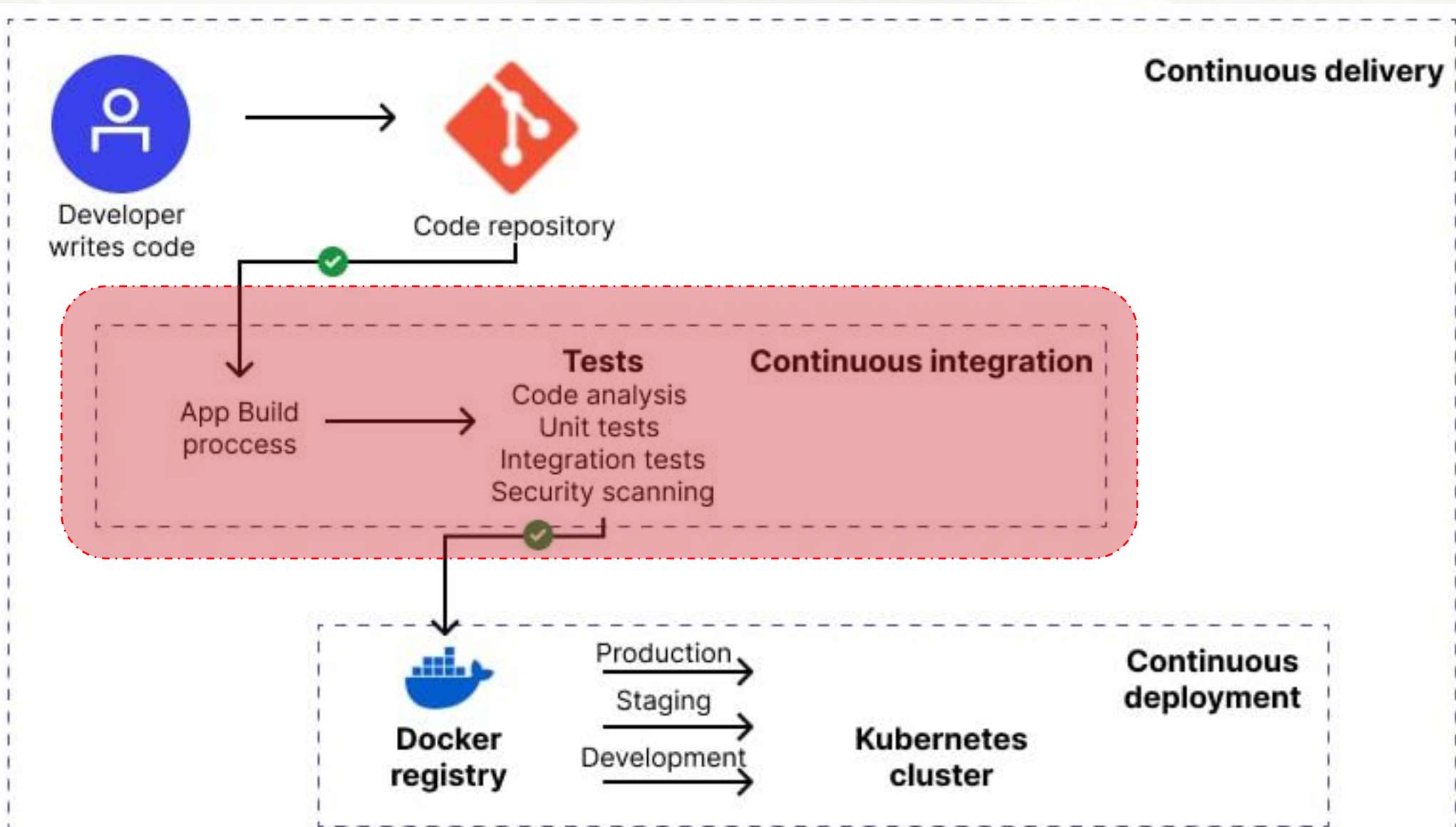
How CI/CD pipeline works!





02

CI





- **Principles**
 - **Keeping track of source code**
 - **Building course mechanization**
 - **Producing a self-checking shape**
 - **Consistent daily commitment from everyone to the baseline**
 - **A solid groundwork for any baseline commitment**
 - **It's important to maintain a rapid progression of construction**





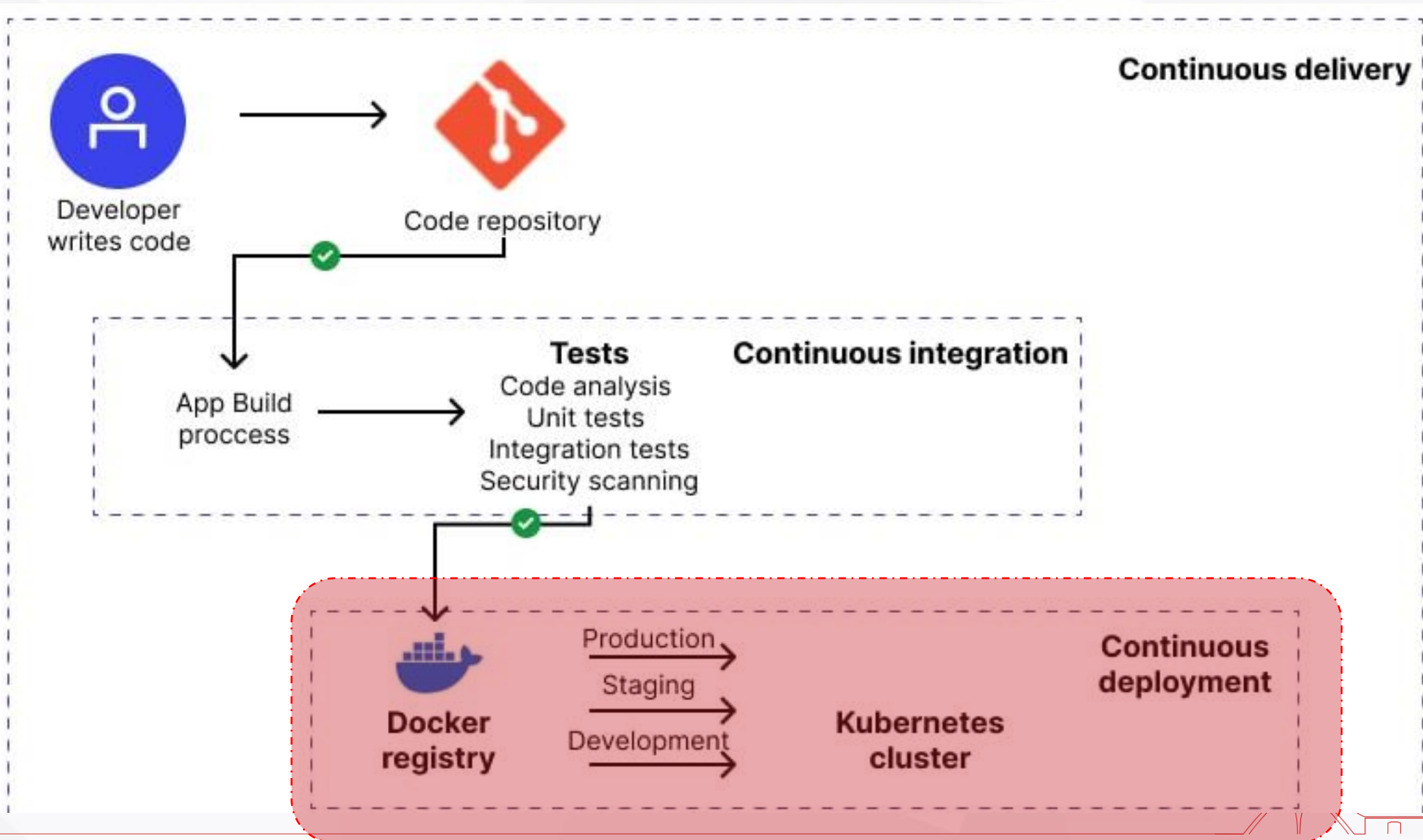
- **Principles**
 - **For quality assurance, it's best to use a replica of the live server**
 - **Facilitate the acquisition of finished products when they become available**
 - **Outcomes of the newest build are visible to all.**
 - **In-automation placement**





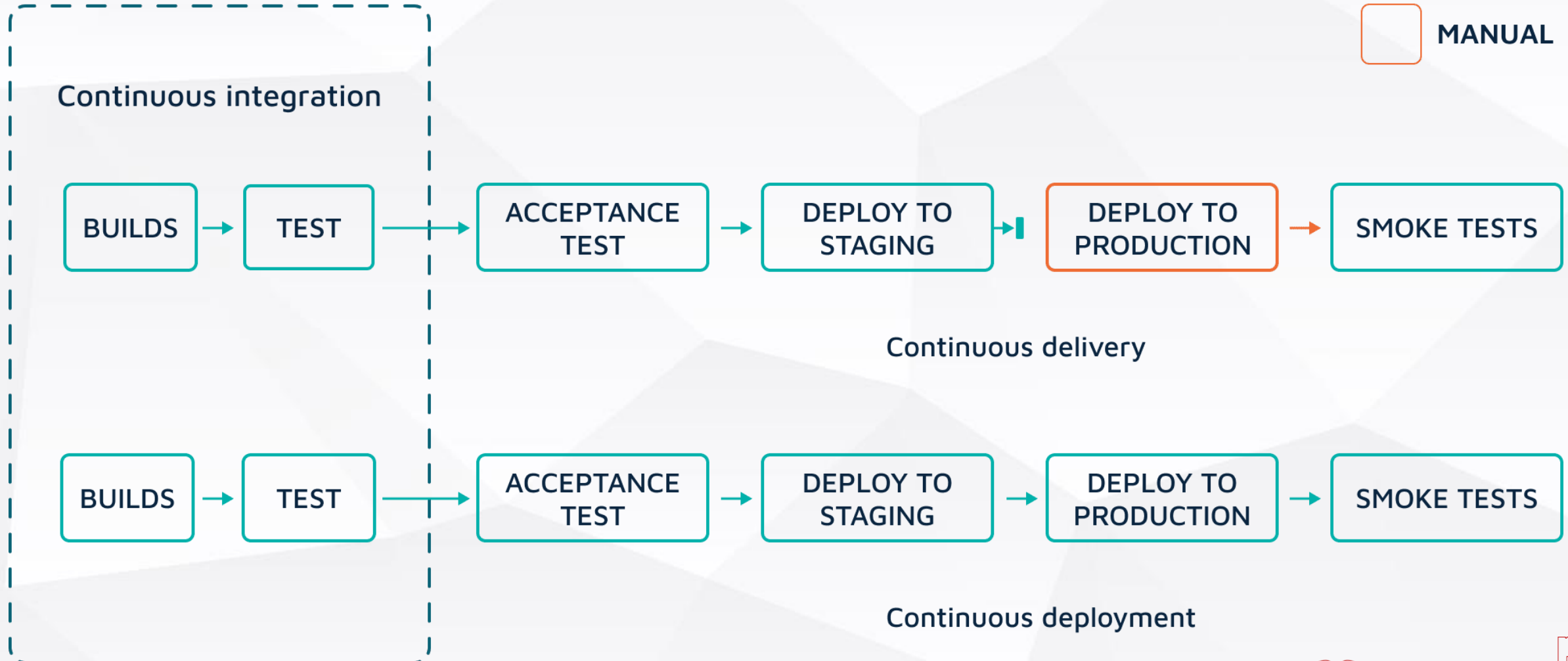
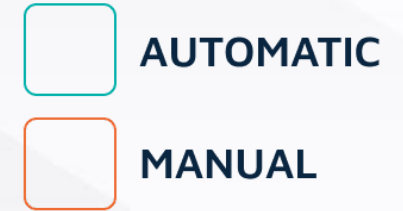
03

CD





Continuous Delivery VS Continuous Deployment





- **Test Automation**
 - **Test automation is the main capability required for continuous deployment. The whole CI/CD chain depends on automation. Development teams can use automated tests to prevent regressions when introducing new code without relying on manual testers to review every code change.**





- **Containerization**
 - **Containerization allows developers to create and deploy applications faster and more securely. It is easier to deploy an application as a container, bundled with all relevant configuration files, libraries, and dependencies needed to run application code**





上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

Non-functional Requirements in Software Engineering

—— 饮水思源 · 爱国荣校 ——

*Beyond “What the system does” → “How
the system works”*





- **What Are Non-functional Requirements (NFRs)?**
 - Constraints or qualities of a system that describe *how it operates*, not what it does.
 - Functional → What the system should do (features, tasks).
 - Non-functional → How the system should be (performance, usability, reliability)
 - **Example:**
 - Functional: “System shall allow users to log in.”
 - Non-functional: “System shall respond to login requests within 2 seconds.”





- **System Architecture**
 - High-level structure of the system, influenced by both FRs and NFRs.
 - Example: Choosing microservices vs monolithic partly depends on scalability (NFR)
- **System Design**
 - Detailed blueprint implementing both FRs and NFRs
- **FRs**
 - Drive module design
- **NFRs**
 - Affect design decisions such as caching, load balancing, encryption





Why Are NFRs Important?

- Determine *user satisfaction* and *system quality*
- Impact *architecture* and *design choices*
- Guide *testing* and *validation* of quality attributes
- Often cause project failure if neglected





Categories of NFRs (Overview)

- **Performance**
- **Usability**
- **Reliability**
- **Security**
- **Maintainability**
- **Portability**
- **Scalability**



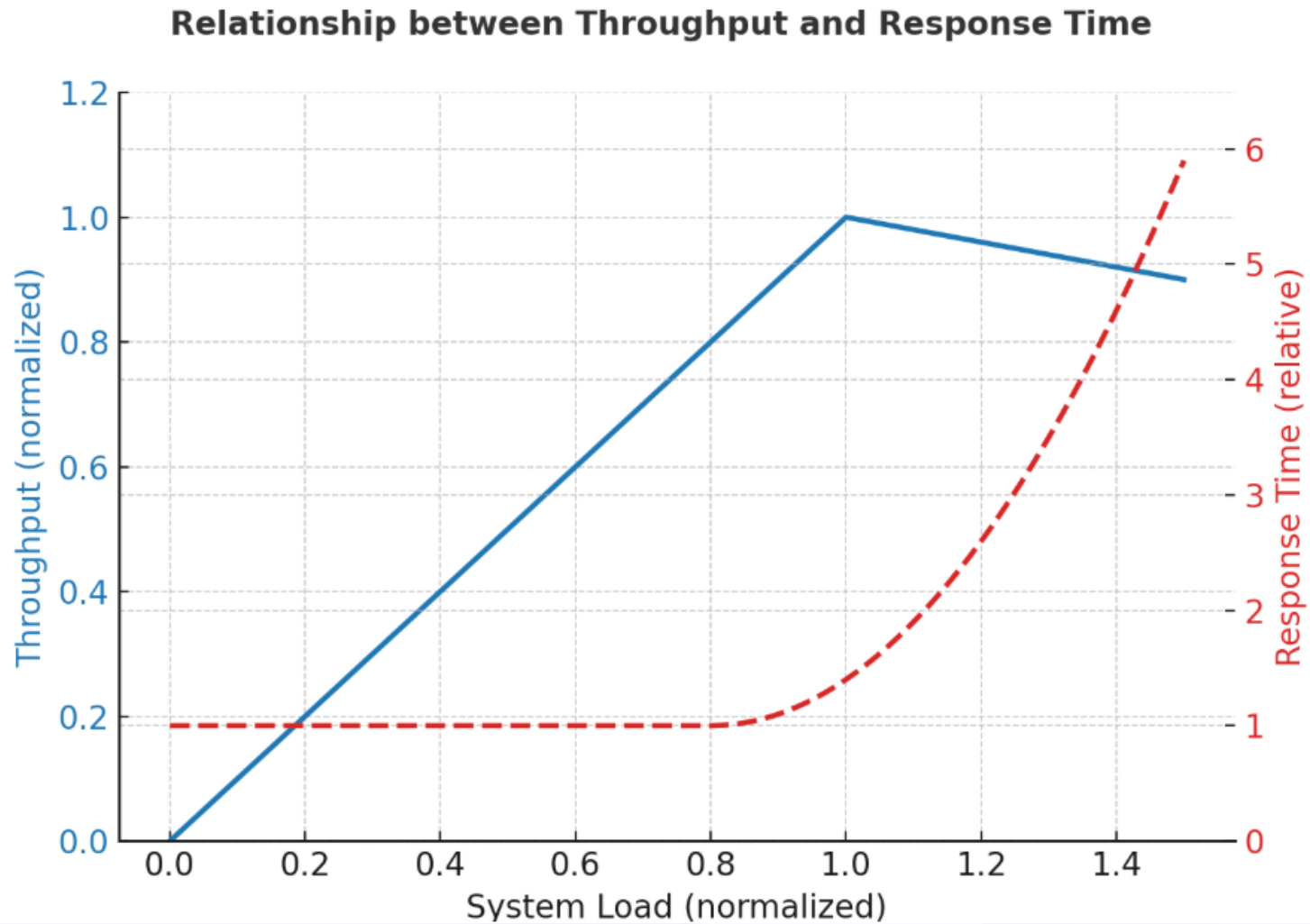


- **Define**
 - *Speed, response time, throughput, resource usage*
- **Examples:**
 - “System shall handle 1,000 transactions per second.”
 - “Page load time $\leq$ 3 seconds.”
- **Tools**
 - Stress testing, profiling





Performance Requirements





- **Define**
 - Ease of learning, efficiency of use, user satisfaction.
- **Examples:**
 - “New users shall complete registration within 2 minutes without training.”
- **Techniques**
 - User testing, UX evaluation, accessibility standards.





- Define
 - Probability system performs without failure
- Measures
 - **Availability**, mean time to failure (MTTF), fault tolerance
- Example:
 - “System availability shall be 99.99%.”
- Techniques
 - Redundancy, failover mechanisms





- **Define**
 - Protecting system and data from threats.
- **Aspects**
 - Confidentiality, Integrity, Authentication, Authorization.
- **Examples:**
 - “All data in transit must be encrypted (TLS 1.3).”
 - “Users must reset passwords every 90 days.”





- **Define**
 - Ease of updating, debugging, extending system.
- **Examples:**
 - “System modules shall have less than 2000 lines of code each.”
- **Practices**
 - Modular design, clean code, documentation.





Portability Requirements

- **Definition**
 - Ability of a system to run in different environments with minimal changes.
- **Examples:**
 - “System shall run on Windows, Linux, and macOS.”
 - “Mobile app shall support both Android and iOS.”
- **Practices**
 - Cross-platform frameworks, containerization, abstraction layers.





- **Definition**
 - The ability of a system to handle increased load (users, data, transactions) without degrading performance.
- **Types of Scalability:**
 - **Vertical scaling (Scale Up)**
 - Add more resources (CPU, RAM) to a single machine.
 - **Horizontal scaling (Scale Out)**
 - Add more machines to distribute the load.





- **Examples:**
 - “System shall support a 10x increase in users without major redesign.”
 - “Database shall scale horizontally with sharding strategy.”
- **Techniques:**
 - Load balancing
 - Distributed caching
 - Cloud auto-scaling
 - Database partitioning / replication





- **Why Trade-offs Matter**
 - **NFRs often conflict with each other.**
 - **Optimizing one quality may degrade another.**
 - **System design must balance according to priorities.**





- **Common Conflicts**
 - **Performance vs Maintainability**, Highly optimized code is faster but harder to maintain.
 - **Security vs Usability**, Strong authentication improves security but reduces ease of use.
 - **Availability vs Consistency** (CAP theorem in distributed systems), high availability may require sacrificing strict consistency.





- **Common Conflicts**
 - **Scalability vs Cost**, scaling out improves capacity but increases infrastructure cost.
 - **Reliability vs Time-to-Market**, building fault tolerance requires more development time.





- **Approaches to Handle Trade-offs**
 - **Prioritization:** Decide which NFRs are critical for stakeholders
 - **Quality Attribute Scenarios:** Define measurable scenarios for decision-making
 - **Architectural Tactics:** Use design patterns to balance trade-offs (e.g., caching, replication, modularization)
 - **Iterative Testing:** Continuously validate NFRs under realistic conditions





How to Specify NFRs

- Use measurable criteria (***SMART: Specific, Measurable, Achievable, Relevant, Time-bound***).
- Avoid vague terms: “fast,” “secure,” “easy.”
- Example
 - Instead of “fast system,” write “system responds within 1s under load of 1000 users.”





- In designing an online video streaming platform, you must consider both *Performance (low latency)* and *Scalability (support millions of users)*.
 - Explain the *potential conflict* between the above NFRs.
 - Propose an *architectural* tactic that balances them.





上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

Architecture

From building systems to designing ecosystems





What Is Software Architecture



- **Defines system' s structure and components**
- **Describes relationships and design principles**
- **Connects functional and non-functional requirements**
- **Provides blueprint for evolution**
- **Serves as communication medium among teams**





- **Single executable or shared memory system**
- **Tight coupling, centralized logic**
- **Easy to deploy initially**
- **Hard to scale or modify**
- **Example**
 - **early ERP**
 - **desktop web servers**





Client-Server and N-Tier

- **Introduced separation of concerns**
- **Client handles UI, server handles logic**
- **Database separated as data tier**
- **Easier maintenance and collaboration**
- **Foundation for enterprise and web apps**





- **Components communicate via standard interfaces**
- **SOAP, REST APIs, and message brokers**
- **Encouraged loose coupling and reusability**
- **Enabled enterprise integration**
- **Foundation for modern microservices**





- **Billions of users, global distribution**
- **Latency and fault tolerance critical**
- **Stateless service design**
- **Horizontal scalability**
- **CDN and caching layers**





- **12-factor applications**
- **Containerization and orchestration**
- **Declarative infrastructure (IaC)**
- **Observability built-in**
- **Continuous delivery and scaling**





- **Architecture affects deployment pipeline**
- **Collaboration between Dev and Ops**
- **Continuous feedback and observability**
- **Architecture = enabler of agility**





- **System = collection of small, independent services**
- **Each service owns its logic and data**
- **Communication via APIs (REST, gRPC, MQ)**
- **Enables independent scaling and deployment**
- **Foundation of cloud-native architecture**





- **Small, cohesive, and independently deployable units**
- **Decentralized data management**
- **Polyglot freedom (different languages/tech stacks)**
- **Automation-first culture (CI/CD, testing, monitoring)**
- **Built for failure and rapid recovery**





- **Scalability per service**
- **Fault isolation**
- **Independent releases**
- **Team autonomy aligned with business domains**
- **Easier experimentation and rollback**





Microservice: Challenges

- **Distributed complexity and latency**
- **Data consistency and transactions**
- **Versioning and dependency chaos**
- **Cross-service testing and debugging difficulty**
- **Operational overhead and cultural shift**





Microservice: Communication Patterns

- **Synchronous: REST, gRPC**
- **Asynchronous: Message Queue, Event Bus**
- **Event-driven architecture for decoupling**
- **Trade-offs: latency vs reliability**
- **Correlation and tracing essential**





- **Each service owns its database**
- **Avoid shared schemas**
- **Saga pattern for distributed transactions**
- **Event sourcing & CQRS for consistency**
- **Data replication and eventual consistency**





Microservice: Service Discovery & Load Balancing

- **Dynamic registration and lookup (Consul, Eureka)**
- **API gateway as single entry point**
- **Circuit breaker and retry patterns**
- **Traffic management for resilience**
- **Enables auto-scaling and failover**





- **Logs, metrics, and distributed traces**
- **Health checks and dashboards**
- **Monitoring tools (Prometheus, Grafana, Jaeger)**
- **Detect bottlenecks and failures quickly**
- **Observability as design principle**





- **Zero-trust principle**
- **Authentication and authorization (OAuth2, JWT, mTLS)**
- **Secret management and encryption**
- **Security automation via policies**
- **Defense in depth across layers**





Microservice: Deployment Patterns

- **Containers and orchestration (Kubernetes)**
- **Blue-Green and Canary deployments**
- **Service Mesh for traffic routing**
- **Continuous delivery and rollback**
- **Observability integrated in deployment**





- **Data- and compute-intensive pipelines**
- **Separation of training and inference paths**
- **GPU/TPU orchestration required**
- **Integration with APIs and microservices**
- **Real-time prediction and feedback loops**





- **Data as first-class citizen**
- **Feature stores and versioned datasets**
- **Streaming platforms (Kafka, Flink)**
- **Unified APIs for ML integration**
- **Collaboration between data and software teams**





- **Model-as-a-Service (MaaS)**
- **MLOps pipelines for automation**
- **Auto-scaling GPU clusters**
- **Explainability, fairness, and privacy layers**
- **Continuous learning infrastructure**





- **Monolithic**
 - simple, fast start, poor scalability
- **Microservices**
 - modular, scalable, operationally complex
- **Serverless**
 - event-driven, cost-efficient, limited control
- **Trade-offs**
 - simplicity vs flexibility vs cost





- **Architectural Decision Records (ADRs)**
- **Evaluate via quality attributes (performance, security, maintainability)**
- **Prioritize trade-offs**
- **Document rationale and impact**
- **Iterative refinement based on feedback**





感谢聆听

饮水思源 爱国荣校